LAB PROGRAMS

Name: S. Aarathana
Reg No: 1921260021

1. TIC TAC TOE GAME

Aim

To implement the Tic Tac Toe game using Python for two players.

Algorithm

1. Create a 3×3 board.
2. Allow players X and O to take turns.
3. Check each row, column and diagonal for a winner.
4. If all cells are filled without a winner, declare a draw.
5. Display the final result.

Progra
m

```
board = [" "] * 9

def show():
    print(board[0], "|", board[1], "|", board[2])
    print("--+---+--")
    print(board[3], "|", board[4], "|", board[5])
    print("--+---+--")
    print(board[6], "|", board[7], "|", board[8])

def winner(mark):
    lines = [(0,1,2),(3,4,5),(6,7,8),
             (0,3,6),(1,4,7),(2,5,8),
             (0,4,8),(2,4,6)]
    return any(all(board[i] == mark for i in line)
               for line in lines)

player = "X"
turn = 0

while turn < 9:
    show()
    pos = int(input(player + ", position (1-9): ")) - 1

    if pos < 0 or pos > 8 or board[pos] != " ":
        print("Invalid position. Try again.")
        continue

    board[pos] = player
    turn += 1

    if winner(player):
        show()
        print(player, "wins!")
        break

    player = "O" if player == "X" else "X"
else:
    show()
    print("It's a draw!")
```

Output

```
========== RESTART: D:/SLO
   |   |
--+---+--
   |   |
--+---+--
   |   |
X, position (1-9): 1
X |   |
--+---+--
   |   |
--+---+--
   |   |
O, position (1-9): 4
X |   |
--+---+--
O |   |
--+---+--
   |   |
X, position (1-9): 2
X | X |
--+---+--
O |   |
--+---+--
   |   |
O, position (1-9): 5
X | X |
--+---+--
O | O |
--+---+--
   |   |
X, position (1-9): 3
X | X | X
--+---+--
O | O |
--+---+--
   |   |
X wins!
```

Result

Thus, the Tic Tac Toe game was implemented successfully.

2. MINIMAX ALGORITHM FOR GAMING

Aim

To implement the Minimax algorithm to choose the best move in a game tree.

Algorithm

1. Represent the game as a tree of possible moves. 2. Give each terminal state a score.

3. At MAX levels, choose the highest score.

4. At MIN levels, choose the lowest score.

5. Display the best score.

Program

```
def minimax(tree, maximizing):
    if not isinstance(tree, list):
        return tree

    scores = [minimax(child, not maximizing) for child in tree]

    return max(scores) if maximizing else min(scores)

game_tree = [[3, 5, 2], [9, 1, 4], [6, 7, 8]]

best_score = minimax(game_tree, True)

print("Game tree:", game_tree)
print("Best score for MAX:", best_score)
```

Output

```
====================== RESTART: D:/SLOT-2/AI/Lab
Game tree: [[3, 5, 2], [9, 1, 4], [6, 7, 8]]
Best score for MAX: 6
>
```

Result

Thus, the Minimax algorithm was implemented to select the best score for the maximizing player.

3. ALPHA-BETA PRUNING ALGORITHM

Aim

To implement Alpha-Beta pruning to reduce the number of game-tree nodes evaluated by Minimax.

Algorithm

1. Initialize alpha to negative infinity and beta to positive infinity.
2. Evaluate child nodes recursively.
3. At MAX nodes, update alpha with the maximum score.
4. At MIN nodes, update beta with the minimum score.
5. Stop checking remaining children when alpha is greater than or equal to beta.

Program

```
def alphabeta(tree, alpha, beta, maximizing):
    if not isinstance(tree, list):
        return tree

    if maximizing:
        value = float("-inf")

        for child in tree:
            value = max(value, alphabeta(child, alpha, beta, False))
            alpha = max(alpha, value)

            if alpha >= beta:
                break

        return value

    else:
        value = float("inf")

        for child in tree:
            value = min(value, alphabeta(child, alpha, beta, True))
            beta = min(beta, value)

            if alpha >= beta:
                break

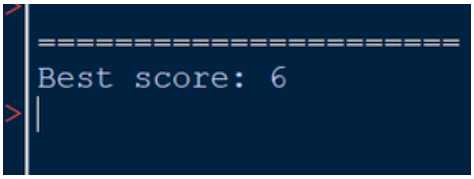
        return value

tree = [[3, 5, 2], [9, 1, 4], [6, 7, 8]]

score = alphabeta(tree, float("-inf"), float("inf"), True)

print("Best score:", score)
```

Output



Result

Thus, Alpha-Beta pruning was implemented to avoid evaluating unnecessary branches.

4. DECISION TREE

Aim

To implement a simple Decision Tree classifier using Python and scikit-learn.

Algorithm

1. Get study hours as input.
2. Check the number of study hours.
3. If hours are 5 or more, predict Pass.
4. Otherwise, predict Fail.
5. Display the result.

Program

```
hours = int(input("Enter study hours: "))

if hours >= 5:
    if hours >= 7:
        result = "Pass with good marks"
    else:
        result = "Pass"
else:
    result = "Fail"

print("Prediction:", result)
```

Output

```
========== RESTART: D:/SLOT-2,
Enter study hours: 6
Prediction: Pass
>>
```

Result

Thus, a Decision Tree classifier was trained and used to predict a result. (Requires scikit-learn.)

5. FEED FORWARD NEURAL NETWORK

Aim

To implement a basic feed-forward neural network using Python and NumPy.

Algorithm

1. Initialize the input, weights and biases.
2. Pass input values through the hidden layer.
3. Apply the sigmoid activation function.
4. Pass hidden-layer values to the output layer.
5. Display the predicted output.

Program

```
def sigmoid(x):
    return 1 / (1 + 2.71828 ** (-x))

x = 0.5
w1 = 0.4
b1 = 0.1

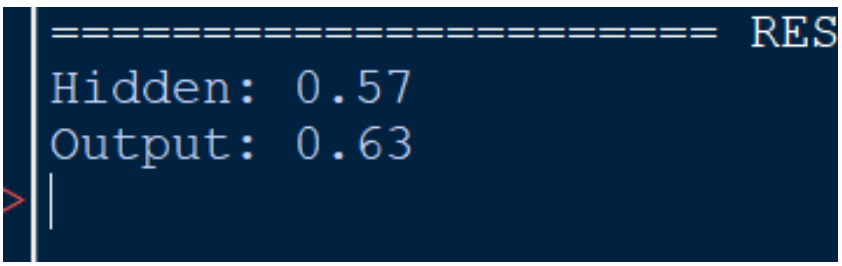
hidden = sigmoid(x * w1 + b1)

w2 = 0.6
b2 = 0.2

output = sigmoid(hidden * w2 + b2)

print("Hidden:", round(hidden, 2))
print("Output:", round(output, 2))
```

Output



Result

Thus, a basic feed-forward neural network was implemented using weighted inputs and sigmoid activation. (Requires NumPy.)